

EIR Records — API Documentation

Resource: `eir_records` **Version:** 2026.06 **Last updated:** June 25, 2026

This reference covers the Equipment Interchange Receipt (EIR) endpoints exposed by the CDMS backend, including request/response shapes, authentication, role requirements, and the recent additions for editing, admin deletion, mandatory photos, and automatic gate fee stamping.

1. Overview

An EIR is the legal record of a container handover between parties. CDMS issues an EIR for every gate-in, gate-out, container sale release, and repatriation movement. Each record is immutable for legal purposes once printed, but may be edited by staff prior to downstream postings and deleted by administrators.

Numbers are minted server-side via the SECURITY DEFINER function `next_eir_number(p_kind text)`, which returns formats such as EIR-GI-000123, EIR-GO-000045, EIR-SLE-000017 (sales release), and EIR-RPT-000009 (repatriation).

2. Authentication

All requests target the PostgREST gateway exposed by Lovable Cloud. Authenticate with a Supabase JWT bearer token in the `Authorization` header and include the project anon key in `apikey`.

```
curl https://<project>.supabase.co/rest/v1/eir_records?select=* \
  -H "apikey: <ANON_KEY>" \
  -H "Authorization: Bearer <USER_JWT>" \
  -H "Accept: application/json"
```

Tenant scoping is enforced server-side: every query is filtered by `current_org_id()` via Row-Level Security. There is no `organization_id` query parameter — the user's active organization is derived from the JWT.

3. Schema

Column	Type	Notes
<code>id</code>	<code>uuid</code>	Primary key, auto-generated.
<code>organization_id</code>	<code>uuid</code>	Tenant scope. Set via trigger.
<code>eir_number</code>	<code>text</code>	Unique. Generated via <code>next_eir_number()</code> .
<code>kind</code>	<code>text</code>	<code>gate_in</code> <code>gate_out</code> <code>sales_release</code> <code>repatriation</code> .
<code>container_id</code>	<code>uuid</code>	FK → <code>containers(id)</code> .
<code>container_number</code>	<code>text</code>	Denormalised for printing.
<code>customer_id</code>	<code>uuid</code>	FK → <code>customers(id)</code> . Nullable.
<code>truck_id</code>	<code>uuid</code>	FK → <code>trucks_drivers(id)</code> . Nullable.
<code>driver_name</code>	<code>text</code>	Captured at gate.
<code>driver_id_number</code>	<code>text</code>	National ID / passport.
<code>truck_plate</code>	<code>text</code>	Plate displayed on EIR.
<code>condition_grade</code>	<code>text</code>	<code>A</code> <code>B</code> <code>C</code> <code>D</code> . Editable.
<code>cargo_status</code>	<code>text</code>	<code>empty</code> <code>laden</code> . Editable.

Column	Type	Notes
seal_number	text	Required for laden. Editable.
damage_description	text	Free text. Editable.
inspector_notes	text	Free text. Editable.
release_purpose	text	sold_unit lease_unit repositioning repair repatriation. (gate_out only)
photos	text[]	Storage paths in container-photos bucket. ≥1 required at gate-in.
gate_fee	numeric	Auto-stamped from active tariff on insert.
gate_fee_currency	text	Auto-filled from organizations.currency.
transport_fee	numeric	Optional, captured at gate-in.
transport_fee_currency	text	Auto-filled.
invoice_id	uuid	Linked customer invoice (if gate fee charged).
printed_at	timestamptz	First print timestamp.
created_by	uuid	auth.users id.
created_at	timestamptz	Insert timestamp.
updated_at	timestamptz	Update timestamp.

4. Endpoints

4.1 GET /rest/v1/eir_records — list

Filter, sort and paginate via PostgREST query params.

```
GET /rest/v1/eir_records
  ?select=id,eir_number,kind,container_number,condition_grade,cargo_status,gate_fee,created_at
  &kind=eq.gate_in
  &order=created_at.desc
  &limit=25
```

200 OK

```
[
  {
    "id": "9f1b...",
    "eir_number": "EIR-GI-000123",
    "kind": "gate_in",
    "container_number": "MSCU1234567",
    "condition_grade": "B",
    "cargo_status": "empty",
    "gate_fee": 1500.00,
    "created_at": "2026-06-25T08:14:22Z"
  }
]
```

4.2 GET /rest/v1/eir_records?id=eq.{id} — fetch one

```
GET /rest/v1/eir_records?id=eq.9f1b...&select=*
Accept: application/vnd.pgrst.object+json
```

4.3 POST /rest/v1/eir_records — create (gate-in)

Photos array must contain at least one storage path for kind = gate_in. eir_number is minted client-side by calling the next_eir_number RPC first.

```
POST /rest/v1/eir_records
Content-Type: application/json
Prefer: return=representation
```

```
{
  "eir_number": "EIR-GI-000124",
  "kind": "gate_in",
  "container_id": "c12...",
  "container_number": "MSCU1234567",
  "customer_id": "cu1...",
  "driver_name": "John Otieno",
  "driver_id_number": "12345678",
  "truck_plate": "KDA 123A",
  "condition_grade": "A",
  "cargo_status": "empty",
  "seal_number": null,
  "damage_description": "",
  "inspector_notes": "Clean unit.",
  "photos": ["org/abc/containers/MSCU1234567/2026-06-25-front.jpg"]
}
```

201 Created — returns the inserted row with gate_fee and currency auto-populated by the tariff trigger.

4.4 PATCH /rest/v1/eir_records?id=eq.{id} — edit

Editable by staff via the in-app pencil dialog. Allowed fields: condition_grade, cargo_status, seal_number, damage_description, inspector_notes, release_purpose (gate_out only).

```
PATCH /rest/v1/eir_records?id=eq.9f1b...
Content-Type: application/json
Prefer: return=representation
```

```
{
  "condition_grade": "C",
  "damage_description": "Dent on left door panel, 30cm.",
  "inspector_notes": "Flagged for M&R."
}
```

4.5 DELETE /rest/v1/eir_records?id=eq.{id} — admin only

Hard delete. Does **not** automatically reverse linked invoices, ledger entries, or container statuses — handle those separately if needed.

```
DELETE /rest/v1/eir_records?id=eq.9f1b...
```

204 No Content on success.

4.6 RPC: next_eir_number

```
POST /rest/v1/rpc/next_eir_number
{ "p_kind": "gate_in" }
```

→ "EIR-GI-000125"

5. Role Requirements

Action	admin	gate_clerk	yard_operator	viewer	customer
List own-org EIRs	✓	✓	✓	✓	own only
Create gate-in	✓	✓	✓	—	—
Create gate-out	✓	✓	✓	—	—
Edit EIR fields	✓	✓	✓	—	—

Action	admin	gate_clerk	yard_operator	viewer	customer
Delete EIR	✓	—	—	—	—
Print / share EIR	✓	✓	✓	✓	own only

RLS summary: SELECT / INSERT / UPDATE policies require `organization_id = current_org_id()`. The DELETE policy additionally requires `has_role(auth.uid(), 'admin')`. Customer-portal users are scoped to `customer_id = current_portal_customer_id()`.

6. Error Responses

Status	When	Example body
401	Missing or expired JWT	<code>{"message":"JWT expired"}</code>
403	RLS denied (wrong org / insufficient role)	<code>{"message":"new row violates row-level security policy"}</code>
404	ID not found in caller's org	<code>[]</code> or <code>{}</code> depending on Accept header
409	Duplicate eir_number	<code>{"code":"23505","message":"duplicate key value"}</code>
422	Photos array empty on gate-in	<code>{"message":"photos required for gate_in"}</code>

7. Recent Changes

- **2026-06:** Added admin-only DELETE via row trash icon (RLS gated by `has_role('admin')`).
- **2026-06:** Added PATCH editing for condition grade, cargo status, seal, damage, notes and release purpose.
- **2026-05:** Mandatory photo upload for gate-in EIRs (UI + server check).
- **2026-05:** Automatic `gate_fee` stamping from active tariff via BEFORE INSERT trigger.
- **2026-05:** Unified numbering via `next_eir_number()` across gate-in, gate-out, sales release, and repatriation kinds.